

**VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING**



**DIGITAL SYSTEM DESIGN
IT105IU**

RISC-V Pipelined Processor Design and FPGA Validation

Submitted by

Tran Thi Ngoc Huyen – ITITI23009

Date Submitted: [04/06/2026]

Date Performed: [04/06/2026]

Course Instructor: Vo Minh Thanh

Table of Contents

1.INTRODUCTION	8
1.1 Background	8
1.2 Problem Statement	8
1.3 Project Objectives	9
1.4 Project Scope.....	9
1.5 Report Organization	10
2.PROCESSING CORE ARCHITECTURE AND CONTROL TOPOLOGY	11
2.1 Introduction	11
2.2 RISC-V Instruction Set Architecture	11
2.2.1 Supported Instruction Set.....	11
2.2.2 R-Type Instructions	11
2.2.3 I-Type Instructions.....	12
2.2.4 S-Type Instructions.....	12
2.2.5 B-Type Instructions	12
2.2.6 J-Type Instructions.....	13
2.3 Overall Processor Architecture	13
2.3.1 Overall Datapath	13
2.3.2 Instruction Execution Flow.....	14
2.4 Core Functional Modules	15
2.4.1 Program Counter	15
2.4.2 Instruction Memory	15
2.4.3 Register File	15
2.4.4 Immediate Generator	16
2.4.5 Arithmetic Logic Unit (ALU).....	16
2.4.6 Data Memory	17
2.4.7 Multiplexers	17
2.4.8 Adders	17
2.5 Control Topology	17
2.5.1 Main Decoder.....	17
2.5.2 ALU Decoder.....	18
2.5.3 Controller Integration.....	18
2.6 Single-Cycle Datapath Operation.....	18

2.6.1	R-Type Execution	18
2.6.2	Load Instruction Execution.....	19
2.6.3	Store Instruction Execution.....	19
2.6.4	Branch Instruction Execution	19
2.6.5	Jump Instruction Execution	19
2.7	Chapter Summary	19
3.PIPELINE ARCHITECTURE, HAZARD UNIT DESIGN AND MEMORY INTEGRATION		20
3.1	Introduction	20
3.2	Five-Stage Pipeline Architecture	20
3.2.1	Pipeline Overview.....	20
3.2.2	Instruction Fetch Stage (IF)	20
3.2.3	Instruction Decode Stage (ID)	20
3.2.4	Execute Stage (EX).....	20
3.2.5	Memory Access Stage (MEM)	21
3.2.6	Write Back Stage (WB)	21
3.3	Pipeline Register Design	21
3.3.1	IF/ID Pipeline Register	21
3.3.2	ID/EX Pipeline Register	21
3.3.3	EX/MEM Pipeline Register	21
3.3.4	MEM/WB Pipeline Register	21
3.4	Hazard Analysis	22
3.4.1	Structural Hazards.....	22
3.4.2	Data Hazards	22
3.4.3	Control Hazards	22
3.5	Forwarding Unit Design.....	22
3.5.1	Forwarding Concept.....	22
3.5.2	Forwarding Sources	22
3.5.3	Forwarding Control Signals.....	23
3.6	Stall Mechanism	23
3.6.1	Load-Use Hazard	23
3.6.2	Stall Control	23
3.7	Control Hazard Resolution.....	23

3.7.1	Branch Hazards	23
3.7.2	Flush Mechanism	23
3.7.3	Pipeline Recovery	23
3.8	Hazard Unit Implementation.....	23
3.9	Memory Integration.....	24
3.9.1	Instruction Memory	24
3.9.2	Data Memory	24
3.9.3	Memory Access Strategy	24
3.9.4	Integration with Pipeline.....	24
3.10	Chapter Summary.....	24
4.	SOFTWARE TOOLCHAIN AND WEB-BASED ASSEMBLER	25
4.1	Introduction	25
4.2	Software Development Workflow	25
4.3	Software Architecture	26
4.3.1	System Structure	26
4.3.2	Software Components	26
4.4	Translation Engine Design.....	27
4.4.1	Register Mapping.....	27
4.4.2	Supported Instruction Set.....	27
4.4.3	Instruction Encoding Methodology	28
4.5	Instruction Translation Process	28
4.5.1	Assembly Parsing.....	28
4.5.2	Operand Decoding	28
4.5.3	Instruction Encoding.....	28
4.5.4	Output Generation.....	28
4.6	Graphical User Interface Design	29
4.6.1	User Interface Overview	29
4.6.2	User Workflow.....	29
4.6.3	GUI Implementation	29
4.7	Error Detection and Validation	30
4.8	Integration with Processor Verification	30
4.9	Advantages of the Developed Tool.....	31
4.10	Chapter Summary.....	31

5. INTEGRATED VALIDATION STRATEGY AND HARDWARE-IN-THE-LOOP TESTING	32
5.1 Introduction	32
5.2 Validation Methodology	32
5.2.1 Functional Simulation	32
5.2.2 Hardware-in-the-Loop Testing	32
5.3 Module-Level Verification.....	32
5.3.1 ALU Verification	32
5.3.2 Control Unit Verification.....	33
5.3.3 Hazard Unit Verification.....	34
5.4 Processor-Level Verification.....	36
5.4.1 Assembly Program Validation.....	36
5.4.2 Pipeline Verification	36
5.5 FPGA Hardware-in-the-Loop Testing	36
5.5.1 FPGA Setup	36
5.5.2 Real hardware Verification	36
5.6 Validation Summary	37
5.7 Chapter Summary.....	37
6. EXPERIMENTAL EVIDENCE, PERFORMANCE WAVEFORMS AND SYSTEM APPRAISAL	38
6.1 Introduction	38
6.2 FPGA Hardware Validation Setup.....	38
6.2.1 Clock Source Selection	38
6.2.2 Display Signal Selection	38
6.3 Experimental Results.....	39
6.4 Comparison with Simulation Results.....	50
6.5 System Appraisal.....	51
6.6 Limitations and Future Enhancements	51
6.7 Chapter Summary.....	51
7. CONCLUSION AND FUTURE WORK	52
7.1 Conclusion.....	52
7.2 Project Contribution	52
7.3 Future Work	52

7.3.1	Expanded RISC-V Instruction Support	52
7.3.2	Branch Prediction Mechanism	53
7.3.3	Cache Memory Integration	53
7.3.4	Advanced Debugging Interface	53
7.3.5	Enhanced Software Toolchain	53
7.3.6	Performance Optimization	53
7.4	Final Remarks	53
8.	REFERENCES.....	54

Table of Figures

Figure 2.1:	Format of R-Type Instruction	12
Figure 2.2:	Format of I-Type Instruction	12
Figure 2.3:	Format of S-Type Instruction	12
Figure 2.4:	Format of B-Type Instruction	13
Figure 2.5:	Format of J-Type Instruction	13
Figure 2.6:	Single-cycle complete datapath	14
Figure 4.1:	Overall Software Development Workflow	26
Figure 4.2:	Universal RISC-V (RV32I) Bitfield Parsing Workspace developed for assembly translation and instruction verification.	30
Figure 5.1:	Waveform for ALU decoder testbench.....	33
Figure 5.2:	Control unit for pipelined processor	34
Figure 5.3:	Waveform for control unit testbench	34
Figure 5.4:	Hazard unit RTL viewer	35
Figure 5.5:	Waveform for hazard testbench	35
Figure 6.1:	1 st cycle of combination set.....	39
Figure 6.2:	2 nd cycle of combination set	40
Figure 6.3:	3 rd cycle of combination set	41
Figure 6.4:	4 th cycle of combination set.....	42
Figure 6.5:	5 th cycle of combination set.....	43

Figure 6.6: 6 th cycle of combination set.....	44
Figure 6.7: 7 th cycle of combination set.....	45
Figure 6.8: 8 th of combination set.....	46
Figure 6.9: 9 th and 10 th cycle of combination set.....	47
Figure 6.10: 11 th cycle of combination set	48
Figure 6.11: 27 th cycle of combination set	49
Figure 6.12: 30 th cycle of combination test	50

1. INTRODUCTION

1.1 Background

Modern digital systems rely heavily on efficient processor architectures to execute increasingly complex applications. As software complexity continues to grow, processor performance becomes a critical factor in determining overall system capability. One of the most influential techniques used to improve processor performance is instruction pipelining, which enables multiple instructions to be processed simultaneously by dividing instruction execution into several stages.

The Reduced Instruction Set Computer (RISC) design philosophy simplifies processor hardware by employing a small and efficient set of instructions. Among modern RISC architectures, RISC-V has gained significant attention due to its open-source nature, modular design, and growing adoption in both academia and industry. The architecture provides an excellent platform for studying processor design principles and implementing customized computing systems.

A pipelined processor offers substantial performance improvements over a traditional single-cycle processor by increasing instruction throughput. However, pipelining introduces challenges such as data hazards and control hazards, which may lead to incorrect execution if not handled properly. Therefore, additional mechanisms such as forwarding, stalling, and flushing are required to maintain correct program execution. Furthermore, developing and testing processor architectures often requires machine-code programs. Manually translating assembly instructions into machine code is time-consuming and prone to human error. To address this issue, software tools such as assemblers are commonly used to automate instruction encoding and simplify processor verification.

This project focuses on the design and implementation of a 32-bit RISC-V pipelined processor featuring hazard detection and resolution mechanisms. In addition, a web-based assembler tool is developed to translate RISC-V assembly programs into machine code compatible with the processor. The complete system is verified through simulation and implemented on an FPGA platform for hardware validation.

1.2 Problem Statement

Single-cycle processors execute each instruction within a single clock period. Although their architecture is relatively simple, their performance is limited because the clock period must accommodate the longest instruction execution path. As a result, hardware resources are not efficiently utilized, leading to reduced throughput.

Pipeline architecture improves performance by allowing multiple instructions to be processed concurrently. However, overlapping instruction execution introduces several hazards. Data hazards occur when an instruction requires data that has not yet been produced by a previous instruction. Control hazards arise when branch or jump instructions change the flow of execution before the next instruction address is determined.

Without proper hazard management techniques, processor correctness and reliability cannot be guaranteed. Therefore, an effective hazard detection and resolution unit must be designed to support pipelined execution.

Another challenge encountered during processor development is the generation of machine code programs for simulation and testing. Manual conversion of assembly instructions into hexadecimal machine code is inefficient and susceptible to encoding mistakes. Consequently, an automated assembly translation tool is desirable to streamline software development and improve testing accuracy.

This project addresses these challenges by developing a pipelined RISC-V processor with comprehensive hazard handling mechanisms and an integrated web-based assembler for machine-code generation.

1.3 Project Objectives

The main objective of this project is to design, implement, and validate a 32-bit RISC-V pipelined processor with hazard resolution capabilities and supporting software tools.

The specific objectives are as follows:

1. To study the RISC-V instruction set architecture and processor design principles.
2. To design and implement the core components of a 32-bit RISC-V processor, including the ALU, register file, control unit, and memory modules.
3. To construct a functional single-cycle processor and extend it into a five-stage pipelined architecture.
4. To develop a hazard unit capable of resolving data and control hazards through forwarding, stalling, and flushing mechanisms.
5. To create a web-based assembler that converts RISC-V assembly instructions into hexadecimal machine code.
6. To verify the correctness of individual modules and the complete processor through simulation testbenches.
7. To implement the processor on an FPGA development board and perform hardware validation.
8. To evaluate the effectiveness of the processor and hazard management mechanisms through experimental testing.

1.4 Project Scope

This project focuses on the design, implementation, and verification of a 32-bit RISC-V pipelined processor and a supporting web-based assembler tool.

The project scope includes:

- Design of a single-cycle RISC-V processor.
- Development of a five-stage pipelined architecture.
- Implementation of instruction fetch, decode, execute, memory access, and write-back stages.
- Support for a subset of RV32I instructions, including arithmetic, logical, load/store, branch, and jump operations.

- Development of hazard resolution techniques including forwarding, stalling, and flushing.
- Creation of a web-based assembler for assembly-to-machine-code translation.
- Functional verification using simulation testbenches.
- FPGA implementation and hardware testing.

The following features are beyond the scope of this project:

- Cache memory systems.
- Branch prediction algorithms.
- Floating-point instruction support.
- Out-of-order execution.
- Superscalar architectures.
- Multi-core processor implementation.
- Full RV32I instruction support.

1.5 Report Organization

This report is organized into seven chapters.

Chapter I introduces the background, problem statement, objectives, scope, and overall organization of the project.

Chapter II presents the processing core architecture and control topology, including the RISC-V instruction formats, processor datapath, control logic, and operation of the single-cycle processor.

Chapter III discusses the design of the pipelined architecture, hazard detection and resolution mechanisms, pipeline registers, and memory integration.

Chapter IV describes the software toolchain, including the design and implementation of the web-based assembler, verification methodology, and hardware-in-the-loop testing strategy.

Chapter V presents the experimental results obtained from simulation, processor verification, assembler validation, and FPGA implementation, together with performance waveforms and execution analysis.

Chapter VI evaluates the overall system performance, analyzes the effectiveness of hazard resolution mechanisms, discusses design limitations, and explores possible architectural improvements.

Chapter VII concludes the project by summarizing the key achievements and presenting recommendations for future work.

2. PROCESSING CORE ARCHITECTURE AND CONTROL TOPOLOGY

2.1 Introduction

This chapter presents the architecture and control topology of the proposed 32-bit RISC-V processor. The processor is initially developed as a single-cycle architecture to establish the fundamental datapath and control mechanisms before being extended into a pipelined design in the following chapter. The architecture supports a subset of the RV32I instruction set and consists of several functional modules including the Program Counter, Instruction Memory, Register File, Arithmetic Logic Unit, Data Memory, Immediate Generator, and Control Unit.

In addition to describing the datapath structure, this chapter presents the control signal generation process, instruction execution flow, RTL designs, and truth tables used to verify the correctness of the processor operation.

2.2 RISC-V Instruction Set Architecture

The processor is based on a subset of the RV32I instruction set architecture. Only the instructions required for arithmetic operations, memory access, branching, and program control are implemented.

2.2.1 Supported Instruction Set

The processor supports the following instructions:

Category	Instructions
Arithmetic	add, sub
Logical	and, or
Comparison	slt
Immediate	addi
Memory Access	lw, sw
Branch	beq
Jump	jal

2.2.2 R-Type Instructions

R-type instructions perform register-to-register operations.

Supported instructions:

- add
- sub
- and
- or
- slt

General format:

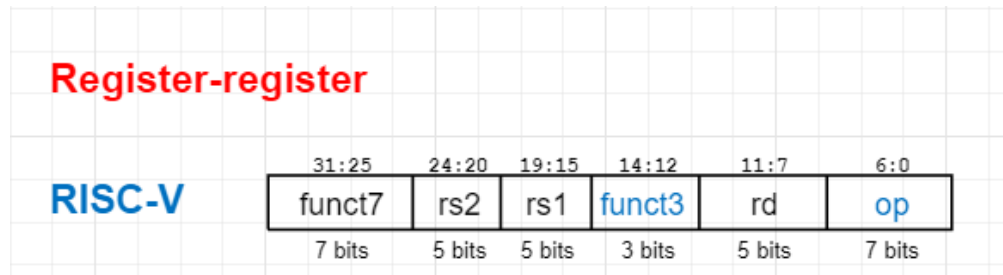


Figure 2.1: Format of R-Type Instruction

2.2.3 I-Type Instructions

I-type instructions use an immediate operand.

Supported instructions:

- addi
- lw

General format:

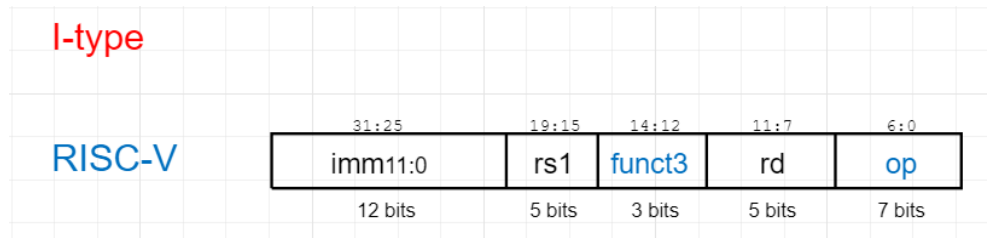


Figure 2.2: Format of I-Type Instruction

2.2.4 S-Type Instructions

S-type instructions are used for memory store operations.

Supported instruction:

- sw

General format:

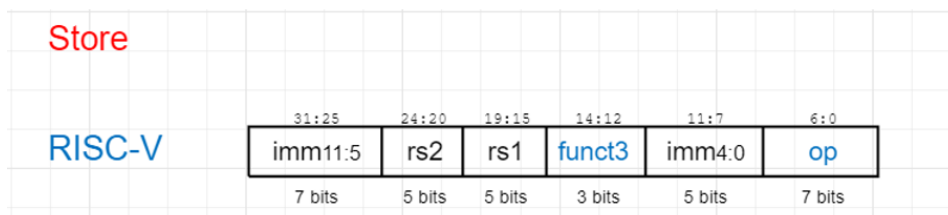


Figure 2.3: Format of S-Type Instruction

2.2.5 B-Type Instructions

B-type instructions perform conditional branching.

Supported instruction:

- beq

General format:

Branch

RISC-V

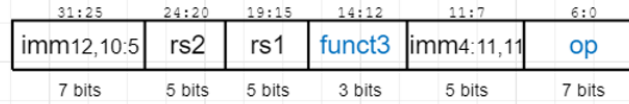


Figure 2.4: Format of B-Type Instruction

2.2.6 J-Type Instructions

J-type instructions support program jumps.

Supported instruction:

- jal

General format:

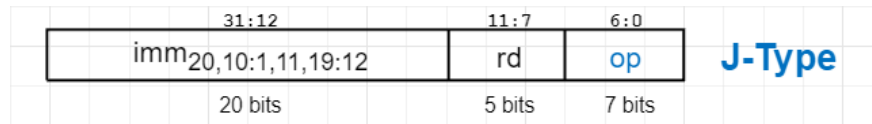


Figure 2.5: Format of J-Type Instruction

2.3 Overall Processor Architecture

2.3.1 Overall Datapath

Figure 2.6 illustrates the complete single-cycle datapath of the proposed processor.

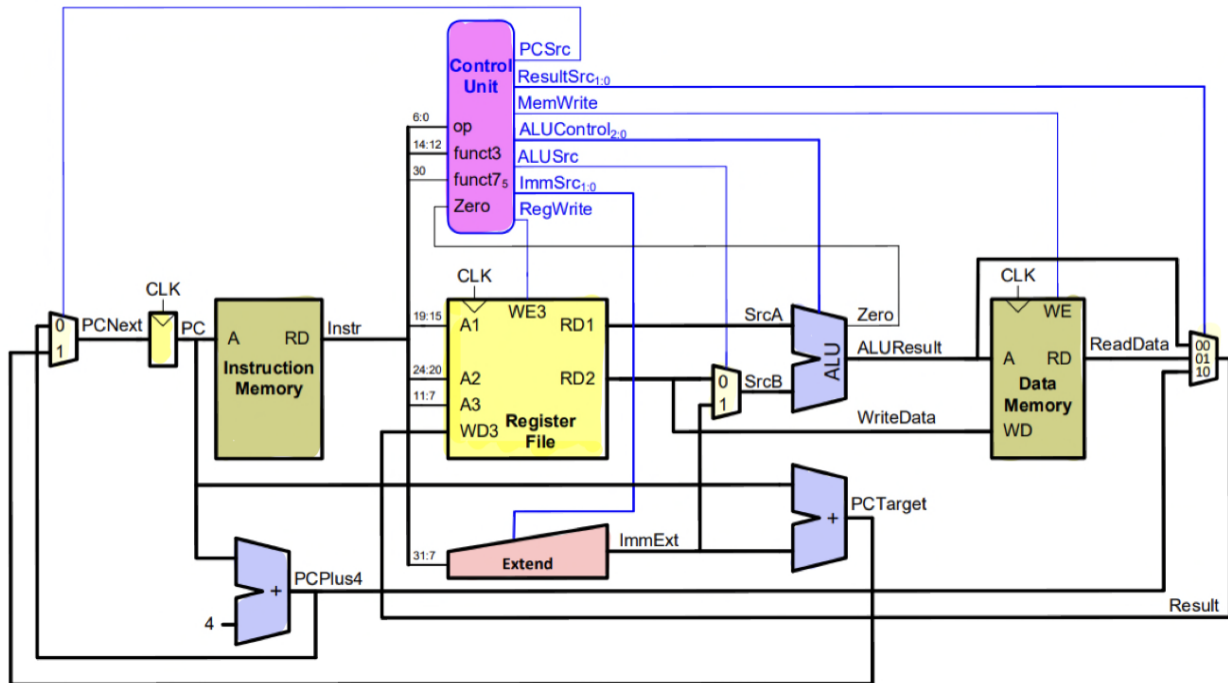


Figure 2.6: Single-cycle complete datapath

The architecture consists of the following major modules:

- Program Counter (PC)
- Instruction Memory
- Register File
- Immediate Generator
- Arithmetic Logic Unit (ALU)
- Data Memory
- Control Unit
- Multiplexers
- Adders

The processor executes each instruction within a single clock cycle. During every clock period, the instruction is fetched, decoded, executed, and written back if required.

2.3.2 Instruction Execution Flow

The instruction execution process consists of:

1. Instruction Fetch
2. Instruction Decode
3. Operand Read
4. Execute
5. Memory Access
6. Write Back

2.4 Core Functional Modules

2.4.1 Program Counter

The Program Counter (PC) stores the address of the current instruction being executed.

Functions

- Holds current instruction address
- Increments by 4 during sequential execution
- Updates with branch/jump target addresses

Inputs

Signal	Description
Clk	System clock
Rst	Reset signal
NextPC	Next instruction address

Output:

Signal	Description
PC	Current instruction address

2.4.2 Instruction Memory

The Instruction Memory stores machine-code instructions generated by the web-based assembler.

Functions

- Fetch instruction according to PC address
- Provide 32-bit instruction to decoder

Inputs and Outputs

Signal	Direction
Address	Input
Instruction	Output

2.4.3 Register File

The Register File contains 32 general-purpose registers.

Functions

- Read two source operands
- Write one destination operand

Inputs

- rs1
- rs2
- rd
- WriteData
- RegWrite

Outputs

- ReadData1

- ReadData2

Register Organization

- 32 Registers
- 32-bit Width
- x0 permanently tied to zero

Truth Table

RegWrite	Operation
0	Read Only
1	Write Enable

2.4.4 Immediate Generator

Function

The Immediate Generator extracts and sign-extends immediate values from instruction fields.

Supported formats

Type	Immediate Width
I-Type	12-bit
S-Type	12-bit
B-Type	13-bit
J-Type	21-bit

Output

- 32-bit immediate value

2.4.5 Arithmetic Logic Unit (ALU)

Function

The ALU performs arithmetic and logical operations.

Inputs

- SrcA
- SrcB
- ALUControl

Outputs

- ALUResult
- Zero Flag

ALU Control Truth Table

ALUControl	Operation
000	ADD
001	SUB
010	AND
011	OR
100	SLT

Test Cases

Test Case	SrcA	SrcB	Expected Result
TC-ALU-01	5	3	8
TC-ALU-02	5	3	2
TC-ALU-03	5	3	1

2.4.6 Data Memory

Function

The Data Memory stores and retrieves data during load and store operations.

Memory Operations

Instruction	Action
Lw	Read
sw	Write

Inputs

- Address
- WriteData
- MemWrite

Output

- ReadData

2.4.7 Multiplexers

Several multiplexers are used throughout the datapath.

Functions

- Select ALU operands
- Select write-back source
- Select next PC value

2.4.8 Adders

Dedicated adders are used to compute:

- PC + 4
- Branch target address
- Jump target address

2.5 Control Topology

The control unit generates control signals required by the datapath modules.

The controller consists of two submodules:

1. Main Decoder
2. ALU Decoder

The opcode field is first decoded by the Main Decoder to generate high-level control signals. The ALU Decoder then determines the specific ALU operation based on instruction type and function fields.

2.5.1 Main Decoder

Generated control signals:

- RegWrite
- MemWrite
- ResultSrc
- ALUSrc
- Branch
- Jump
- ImmSrc
- ALUOp

Control Signal Truth Table

Instruction	RegWrite	ALUSrc	MemWrite	ResultSrc	Branch	Jump
add	1	0	0	ALU	0	0
sub	1	0	0	ALU	0	0
lw	1	1	0	MEM	0	0
sw	0	1	1	X	0	0
beq	0	0	0	X	1	0
jal	1	X	0	PC+4	0	1

2.5.2 ALU Decoder

The ALU Decoder generates:

- ADD
- SUB
- AND
- OR
- SLT

control signals for the ALU.

ALU Decoder Truth Table

ALUOp	funct3	funct7	ALUControl
00	XXX	XXXXXXXX	ADD
01	XXX	XXXXXXXX	SUB
10	000	0000000	ADD
10	000	0100000	SUB
10	111	0000000	AND
10	110	0000000	OR
10	010	0000000	SLT

2.5.3 Controller Integration

The outputs from both decoders are combined to control the entire datapath and ensure correct execution of each instruction.

2.6 Single-Cycle Datapath Operation

2.6.1 R-Type Execution

For arithmetic instructions, operands are read from the register file and processed by the ALU. The result is written back to the destination register.

2.6.2 Load Instruction Execution

The ALU calculates the memory address using a base register and immediate offset. Data is read from memory and written back to the destination register.

2.6.3 Store Instruction Execution

The ALU computes the target memory address and stores the source register value into memory.

2.6.4 Branch Instruction Execution

The ALU compares two source registers. If the branch condition is satisfied, the PC is updated with the branch target address.

2.6.5 Jump Instruction Execution

The jump target address is calculated and loaded into the Program Counter, allowing execution to continue from a new location.

2.7 Chapter Summary

This chapter presented the architecture and control topology of the proposed single-cycle RISC-V processor. The supported instruction formats, datapath components, and control mechanisms were discussed in detail. The single-cycle processor serves as the foundation for the pipelined architecture introduced in the next chapter, where hazard detection and resolution mechanisms are incorporated to improve execution performance.

3. PIPELINE ARCHITECTURE, HAZARD UNIT DESIGN AND MEMORY INTEGRATION

3.1 Introduction

The single-cycle processor presented in Chapter II provides a functional implementation of the selected RISC-V instruction set. However, its performance is limited because each instruction must complete all processing stages within a single clock cycle. To improve instruction throughput and processor efficiency, the architecture is extended into a pipelined design.

This chapter presents the design of the five-stage pipeline architecture, the implementation of pipeline registers, hazard analysis, hazard resolution techniques, and memory integration strategies employed in the proposed processor.

3.2 Five-Stage Pipeline Architecture

3.2.1 Pipeline Overview

The processor adopts a five-stage pipeline architecture consisting of:

1. Instruction Fetch (IF)
2. Instruction Decode (ID)
3. Execute (EX)
4. Memory Access (MEM)
5. Write Back (WB)

By allowing multiple instructions to occupy different stages simultaneously, the pipeline improves instruction throughput compared to the single-cycle architecture.

3.2.2 Instruction Fetch Stage (IF)

The Instruction Fetch stage retrieves instructions from Instruction Memory using the Program Counter.

Responsibilities:

- Instruction retrieval
- Program Counter update
- Sequential address generation

3.2.3 Instruction Decode Stage (ID)

The Instruction Decode stage interprets the instruction and prepares operands for execution.

Responsibilities:

- Instruction decoding
- Register file access
- Immediate generation
- Control signal generation

3.2.4 Execute Stage (EX)

The Execute stage performs arithmetic and logical operations.
Responsibilities:

- ALU operations
- Branch target calculation
- Operand selection

3.2.5 Memory Access Stage (MEM)

The Memory Access stage performs data transfer operations.
Responsibilities:

- Data memory read
- Data memory write

3.2.6 Write Back Stage (WB)

The Write Back stage updates the Register File with the final execution result.
Responsibilities:

- Result selection
- Register update

3.3 Pipeline Register Design

Pipeline registers isolate adjacent stages and store intermediate data between clock cycles.

3.3.1 IF/ID Pipeline Register

Stores:

- Instruction
- Program Counter
- $PC + 4$

3.3.2 ID/EX Pipeline Register

Stores:

- Register operands
- Immediate values
- Source and destination register addresses
- Control signals

3.3.3 EX/MEM Pipeline Register

Stores:

- ALU result
- Memory write data
- Destination register address
- Control signals

3.3.4 MEM/WB Pipeline Register

Stores:

- Memory read data
- ALU result
- Destination register address
- Control signals

3.4 Hazard Analysis

Pipeline execution introduces several types of hazards that may affect processor correctness.

3.4.1 Structural Hazards

Structural hazards occur when multiple instructions compete for the same hardware resource.

The proposed processor avoids structural hazards through separate instruction and data memory paths.

3.4.2 Data Hazards

Data hazards occur when an instruction depends on data produced by a preceding instruction that has not yet completed execution.

Example:

add x1, x2, x3

sub x4, x1, x5

The second instruction requires the result generated by the first instruction.

3.4.3 Control Hazards

Control hazards occur when branch or jump instructions alter the normal execution sequence.

Example:

beq x1, x2, LABEL

The processor must determine whether the branch is taken before continuing execution.

3.5 Forwarding Unit Design

To reduce performance degradation caused by data hazards, a forwarding mechanism is implemented.

3.5.1 Forwarding Concept

Forwarding transfers data directly from later pipeline stages to the Execute stage without waiting for register write-back.

3.5.2 Forwarding Sources

The forwarding unit can obtain data from:

- EX/MEM stage
- MEM/WB stage

3.5.3 Forwarding Control Signals

The forwarding unit generates:

- ForwardAE
- ForwardBE

These signals determine the source of ALU operands during execution.

3.6 Stall Mechanism

Certain hazards cannot be resolved using forwarding alone.

3.6.1 Load-Use Hazard

A load-use hazard occurs when an instruction immediately depends on data being loaded from memory.

Example:

```
lw x1, 0(x2)
```

```
add x3, x1, x4
```

Since the loaded data is not available during the required pipeline stage, the pipeline must be temporarily stalled.

3.6.2 Stall Control

The stall mechanism prevents incorrect execution by:

- Freezing the Program Counter
- Freezing the IF/ID register
- Inserting a bubble into the pipeline

3.7 Control Hazard Resolution

3.7.1 Branch Hazards

Branch instructions may cause incorrect instructions to enter the pipeline before the branch outcome is determined.

3.7.2 Flush Mechanism

The processor resolves branch hazards by flushing invalid instructions from the pipeline whenever a branch or jump is taken.

3.7.3 Pipeline Recovery

After flushing, instruction execution resumes from the correct target address.

3.8 Hazard Unit Implementation

The Hazard Unit coordinates all hazard resolution mechanisms.

Inputs

- Source register addresses
- Destination register addresses
- Register write signals

- Branch control signals

Outputs

- ForwardAE
- ForwardBE
- StallF
- StallD
- FlushD
- FlushE

The Hazard Unit continuously monitors instruction dependencies and generates appropriate control signals to maintain correct execution.

3.9 Memory Integration

3.9.1 Instruction Memory

Instruction Memory stores machine code generated by the web-based assembler and supplies instructions to the Fetch stage.

3.9.2 Data Memory

Data Memory stores variables and supports load/store operations.

3.9.3 Memory Access Strategy

Separate instruction and data memory modules are employed to simplify memory access and eliminate structural hazards.

3.9.4 Integration with Pipeline

The memory subsystem interfaces directly with the IF and MEM stages, enabling simultaneous instruction fetching and data access.

3.10 Chapter Summary

This chapter described the design of the pipelined processor architecture, including the five-stage pipeline organization, pipeline registers, hazard analysis, forwarding mechanisms, stall control logic, control hazard handling, and memory integration strategy. These architectural enhancements enable improved instruction throughput while preserving correct program execution.

4. SOFTWARE TOOLCHAIN AND WEB-BASED ASSEMBLER

4.1 Introduction

The development and verification of a RISC-V processor require executable machine instructions that can be loaded into instruction memory for simulation and FPGA implementation. Although machine code can be manually encoded according to the RV32I instruction specification, the process is tedious and prone to human error.

To simplify program generation and instruction verification, a graphical RISC-V Assembly Translation Tool was developed using Python and the Tkinter framework. The tool automatically converts RV32I assembly instructions into machine code while simultaneously displaying instruction bit-fields and hexadecimal representations. This capability allows users to verify instruction encoding and generate machine code for processor testing.

This chapter presents the architecture, implementation, and integration of the developed software toolchain.

4.2 Software Development Workflow

The software toolchain was designed to automate the conversion of assembly instructions into executable machine code suitable for processor simulation and FPGA implementation.

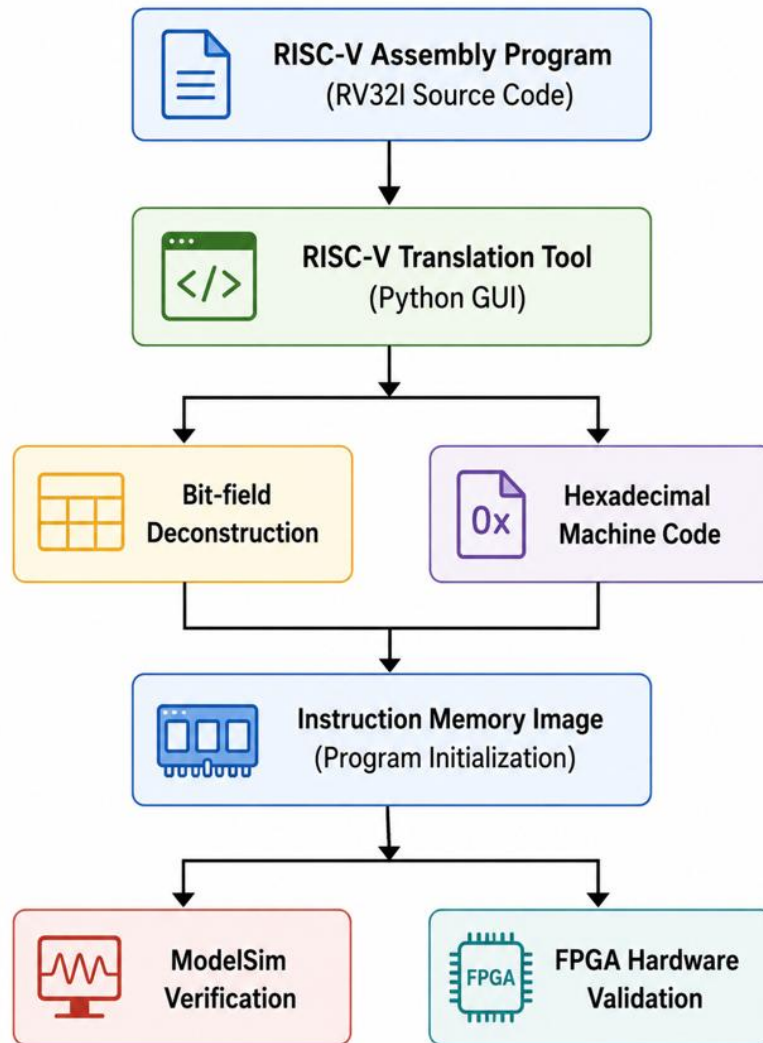


Figure 4.1: Overall Software Development Workflow

4.3 Software Architecture

4.3.1 System Structure

The software consists of two major subsystems:

1. Translation Engine
2. Graphical User Interface

The Translation Engine performs instruction parsing and machine-code generation, while the graphical interface provides an interactive environment for user input and output visualization.

4.3.2 Software Components

The tool contains:

- Register Mapping Module
- Token Parser
- Immediate Value Processor
- Instruction Encoder
- Binary Output Generator
- Hexadecimal Output Generator
- GUI Interface Controller

The interaction among these modules enables automatic conversion of assembly instructions into machine code suitable for processor verification.

4.4 Translation Engine Design

4.4.1 Register Mapping

The translation engine maintains an internal register mapping table corresponding to the RV32I register architecture.

Register	Number
x0 (zero)	0
x1 (ra)	1
x1 (sp)	2
x5 (t0)	5
x6 (t1)	6
x7 (t2)	7
x10 (a0)	10
x11 (a1)	11

The register table enables symbolic register names to be converted into binary register addresses during instruction encoding.

4.4.2 Supported Instruction Set

The current implementation supports the following RV32I instruction categories.

R-Type Instructions

Instruction	Function
add	Addition
sub	Subtraction
and	Logical AND
or	Logical OR
slt	Set Less Than

I-Type Instructions

Instruction	Function
Addi	Add Immediate
lw	Load Word

S-Type Instructions

Instruction	Function
sw	Store Word

B-Type Instructions

Instruction	Function
beq	Branch Equal

U-Type Instructions

Instruction	Function
lui	Load Upper Immediate

J-Type Instructions

Instruction	Function
jal	Jump and Link

4.4.3 Instruction Encoding Methodology

Each instruction is encoded according to its corresponding RV32I format. The translation engine extracts instruction fields including opcode, destination register, source registers, function codes, and immediate values. These fields are assembled into a 32-bit machine instruction and subsequently converted into hexadecimal format.

4.5 Instruction Translation Process

The instruction translation process consists of four major stages.

4.5.1 Assembly Parsing

The parser reads assembly instructions entered by the user and separates each instruction into individual tokens. The instruction mnemonic and operands are extracted for further processing.

4.5.2 Operand Decoding

Register names and immediate values are identified and translated into their numerical representations. The decoder also determines the appropriate instruction format.

4.5.3 Instruction Encoding

The instruction encoder constructs the 32-bit machine instruction by combining opcode fields, register addresses, function fields, and immediate values according to the RV32I specification.

4.5.4 Output Generation

The generated instruction is displayed in two forms:

- Structured bit-field representation
- Hexadecimal machine code representation

This dual-output approach allows users to verify instruction encoding while simultaneously generating machine code for processor execution.

4.6 Graphical User Interface Design

4.6.1 User Interface Overview

The graphical interface was developed using Python Tkinter and provides a user-friendly environment for instruction translation.

The interface consists of:

- Assembly Input Workspace
- Translation Execution Button
- Reset Function
- Bit-field Output Panel
- Hexadecimal Output Panel

4.6.2 User Workflow

The user enters RV32I assembly instructions into the input panel and executes the translation process using the provided control button. The generated bit-fields and hexadecimal machine code are displayed simultaneously in the output panels.

4.6.3 GUI Implementation

Figure 4.2 shows the developed Universal RISC-V (RV32I) Bitfield Parsing Workspace.

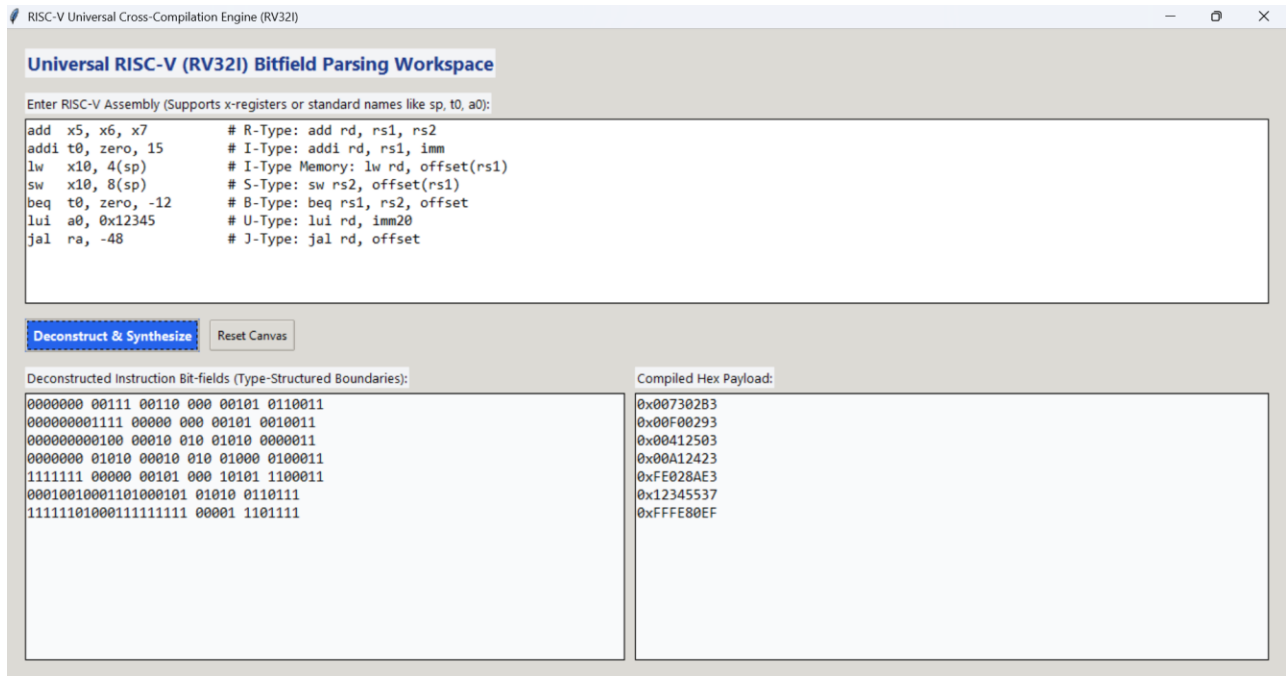


Figure 4.2: Universal RISC-V (RV32I) Bitfield Parsing Workspace developed for assembly translation and instruction verification.

4.7 Error Detection and Validation

The developed tool incorporates error-detection mechanisms to improve reliability. Supported checks include:

- Invalid instruction mnemonics
- Invalid register names
- Missing operands
- Unsupported instruction formats
- Incorrect immediate value syntax
- Parsing failures

Error messages are displayed to assist users in correcting assembly code before processor execution.

4.8 Integration with Processor Verification

The generated machine code is integrated directly into the processor verification workflow.

Applications include:

- Datapath verification
- ALU testing
- Control unit validation
- Hazard unit verification
- Instruction memory initialization
- ModelSim simulation
- FPGA hardware implementation

The generated hexadecimal machine code can be loaded directly into instruction memory, while the bit-field representation assists in debugging instruction encoding and control-signal generation.

4.9 Advantages of the Developed Tool

The developed assembler tool provides several advantages:

- Automatic machine-code generation
- Support for multiple RV32I instruction formats
- Simultaneous bit-field and hexadecimal outputs
- Reduced manual encoding errors
- Faster processor testing and debugging
- User-friendly graphical interface
- Direct integration with simulation and FPGA verification workflows

4.10 Chapter Summary

This chapter presented the software toolchain and assembly translation tool developed for the RV32I processor. The translation engine, instruction encoding process, graphical user interface, and integration with processor verification were discussed. The developed tool simplifies machine-code generation and supports efficient simulation and FPGA validation.

5. INTEGRATED VALIDATION STRATEGY AND HARDWARE-IN-THE-LOOP TESTING

5.1 Introduction

Verification and validation are essential stages in processor development to ensure that all hardware modules operate according to the RV32I specification. The proposed processor was validated using a combination of simulation-based verification and hardware-in-the-loop (HIL) testing on the FPGA platform.

This chapter presents the validation methodology, test scenarios, simulation results, FPGA implementation, and performance evaluation of the developed processor.

5.2 Validation Methodology

The verification strategy consisted of two major phases:

5.2.1 Functional Simulation

Functional simulation was performed using ModelSim to verify:

- Arithmetic operations
- Logic operations
- Branch instructions
- Memory access operations
- Program flow control

5.2.2 Hardware-in-the-Loop Testing

After simulation verification, the design was synthesized and deployed to the FPGA board.

Hardware testing was conducted to verify:

- Real-time processor execution
- Memory access functionality
- Hazard handling mechanisms
- Clock synchronization
- Instruction execution correctness

5.3 Module-Level Verification

5.3.1 ALU Verification

The Arithmetic Logic Unit was tested using different operand combinations.

Test Cases

Operation	Operand A	Operand B	Expected Result
ADD	10	20	30
SUB	20	10	10
AND	15	7	7
OR	8	4	12
SLT	5	10	1

Simulation Results

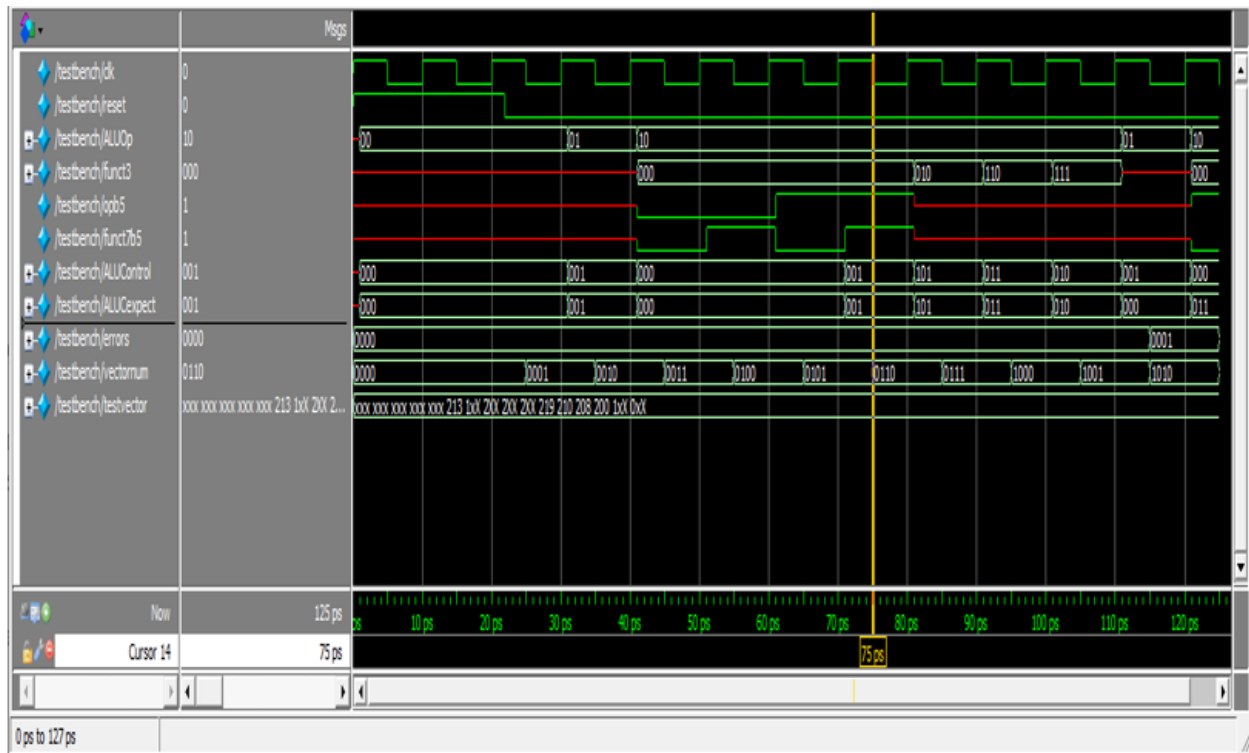


Figure 5.1: Waveform for ALU decoder testbench

5.3.2 Control Unit Verification

The control unit was verified using supported instruction types.

Test Cases

Instruction	Expected Control Signals
add	R-Type Control
addi	I-Type Control
lw	Load Control
sw	Store Control
beq	Branch Control
jal	Jump Control

RTL Verification

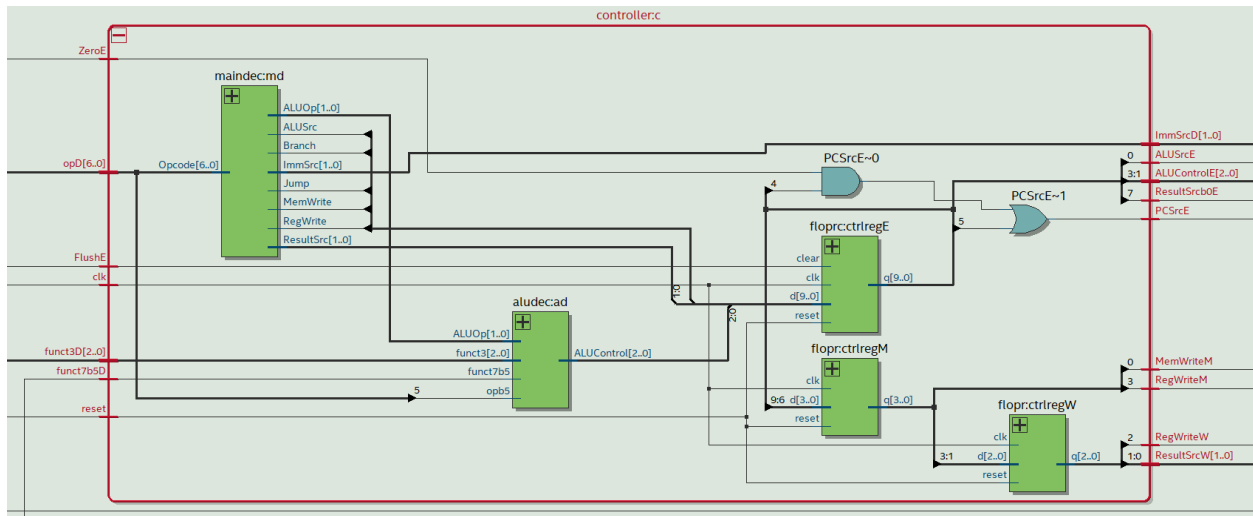


Figure 5.2: Control unit for pipelined processor

Simulation Results

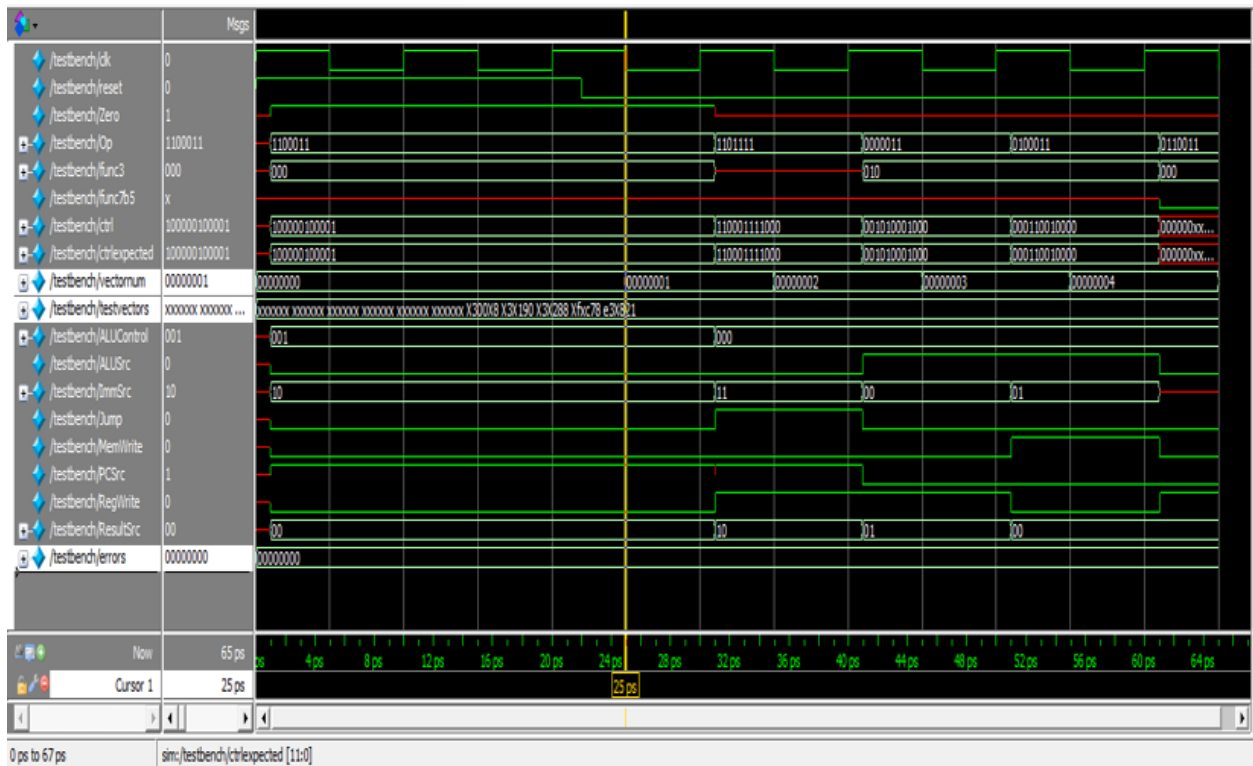


Figure 5.3: Waveform for control unit testbench

5.3.3 Hazard Unit Verification

The Hazard Detection Unit was verified using data dependency scenarios.

Hazard Test Example

addi x5, x0, 10

add x6, x5, x7

Expected behavior

- Pipeline stall generated
- Correct forwarding performed

RTL Verification

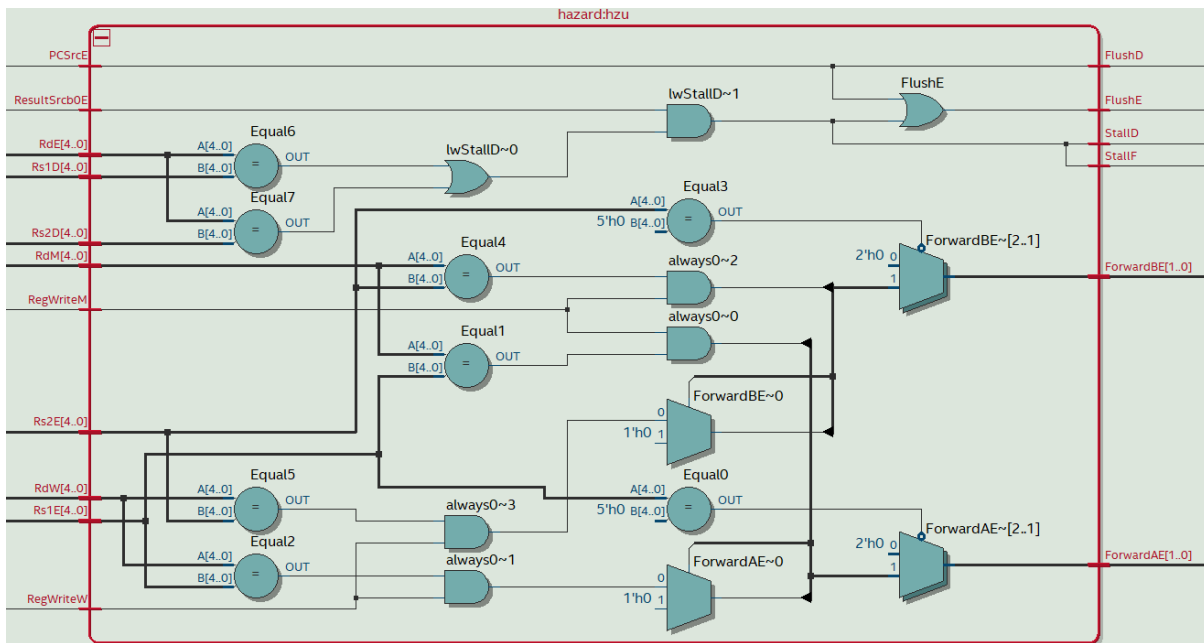


Figure 5.4: Hazard unit RTL viewer

Simulation Results

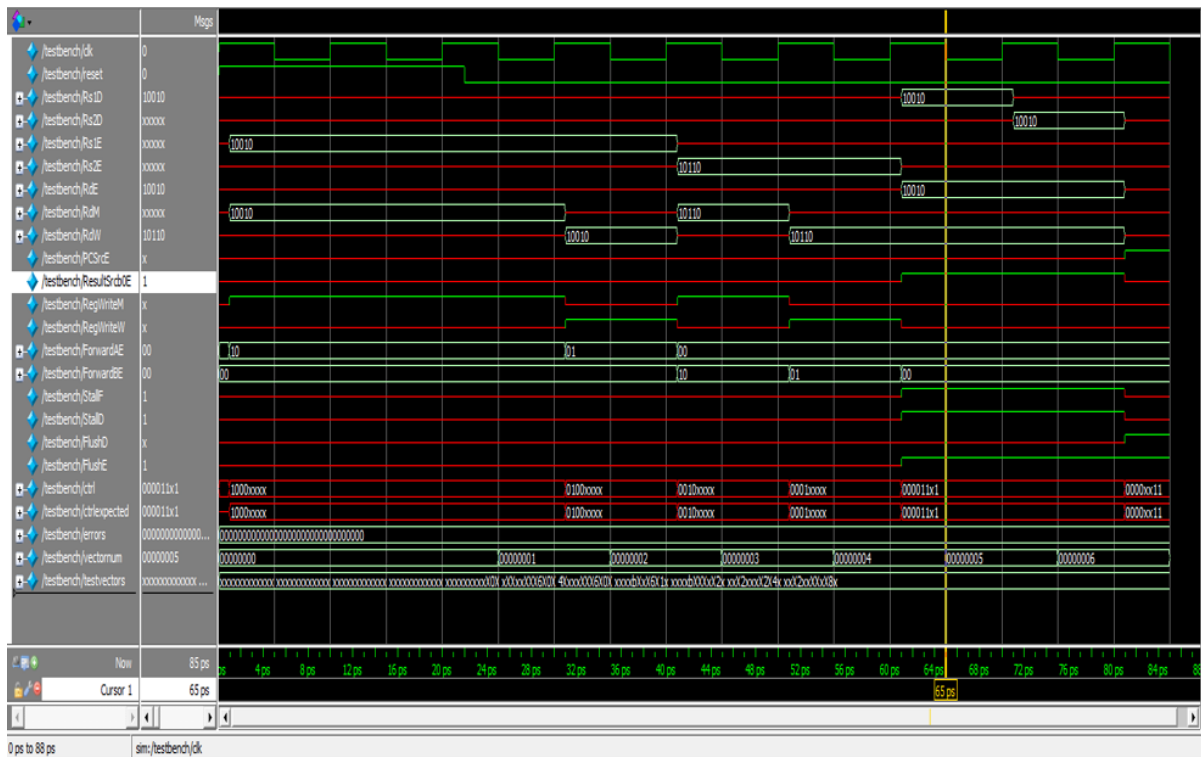


Figure 5.5: Waveform for hazard testbench

5.4 Processor-Level Verification

5.4.1 Assembly Program Validation

The processor was tested using machine code generated by the developed RV32I Assembly Translation Tool.

Test Program

```
addi t0, zero, 15
lw x10, 4(sp)
sw x10, 8(sp)
beq t0, zero, -12
lui a0, 0x12345
jal ra, -48
```

Generated Machine Code

Instruction	Binary	Hexadecimal
addi t0, zero, 15	000000001111 00000 000 00101 0010011	0x00F00293
lw x10, 4(sp)	000000000100 00010 010 01010 0000011	0x00412503
sw x10, 8(sp)	00000000 01010 00010 010 01000 0100011	0x00A12423
beq t0, zero, -12	1111111 00000 00101 000 10101 1100011	0xFE028AE3
lui a0, 0x12345	00010010001101000101 01010 0110111	0x12345537
jal ra, -48	11111101000111111111 00001 1101111	0xFFFE80EF

5.4.2 Pipeline Verification

The complete pipeline was verified using multiple instruction sequences.

Verification objectives:

- Correct instruction fetch
- Correct instruction decode
- Correct ALU execution
- Correct memory access
- Correct write-back operation

5.5 FPGA Hardware-in-the-Loop Testing

5.5.1 FPGA Setup

The processor was implemented on the FPGA development board.

Hardware resources:

- FPGA Device
- On-board Clock
- Switch Inputs
- LED Outputs
- Seven-Segment Displays

5.5.2 Real hardware Verification

Several test programs were executed directly on hardware.

Hardware Test Scenarios

Test	Objective	Result
Arithmetic Program	ALU verification	Pass
Memory Program	Memory verification	Pass
Branch Program	Control verification	Pass
Hazard Program	Hazard verification	Pass

5.6 Validation Summary

The developed RV32I processor successfully passed module-level verification, processor-level validation, and FPGA hardware testing. Simulation waveforms confirmed correct instruction execution, while hardware-in-the-loop testing demonstrated reliable operation on the FPGA platform. The results validate the functionality of the proposed processor architecture and its supporting subsystems.

5.7 Chapter Summary

This chapter presented the validation methodology and testing results for the proposed RV32I processor. Functional simulation, hazard verification, processor-level testing, and FPGA hardware validation were performed. The obtained results confirmed the correctness and reliability of the implemented design.

6. EXPERIMENTAL EVIDENCE, PERFORMANCE WAVEFORMS AND SYSTEM APPRAISAL

6.1 Introduction

After successful simulation-based verification, the proposed RV32I pipelined processor was implemented and tested on the FPGA development board. The objective of the hardware validation stage was to confirm that the processor operates correctly under real execution conditions and that the observed hardware behavior matches the simulation results presented in Chapter V.

To facilitate system verification and debugging, several internal processor signals were connected to the FPGA display interface. By using the FPGA switches, users can select and observe different datapath signals in real time through the seven-segment displays. This approach provides direct hardware evidence of processor functionality and pipeline operation.

6.2 FPGA Hardware Validation Setup

6.2.1 Clock Source Selection

The processor supports both manual and automatic execution modes through switch SW[7]. A clock multiplexer selects either the debounced push-button clock or the internally generated clock signal.

SW[7]	Clock Source	Operating Mode
0	debounce_clk	Manual Execution Mode
1	gen_clk	Automatic Execution Mode

In Manual Execution Mode, the processor advances one clock cycle whenever the push button is pressed. This mode is useful for debugging and observing individual instruction execution.

In Automatic Execution Mode, the processor executes continuously using the internally generated clock signal, allowing uninterrupted program execution.

6.2.2 Display Signal Selection

The displayed processor signal is selected through switch inputs SW[6:0].

Switch Configuration (SW[6:0])	Displayed Signal	Description
0000000	PC_NowE	Program Counter in Execute Stage
0000001	RdE	Destination Register Address
0000010	Rs1E	Source Register 1 Value
0000011	Rs2E	Source Register 2 Value
0000100	PCTargetE	Branch/Jump Target Address

0000101	ReadDataM	Data Memory Read Output
0000110	WriteDataE	Data Written to Memory
0000111	ALU_outE	ALU Execution Result

The selected signal is converted into hexadecimal format and displayed on eight seven-segment displays for real-time monitoring.

6.3 Experimental Results



Figure 6.1: 1st cycle of combination set

As displayed in Figure 6.1, the first clock cycle evaluates an I-type immediate instruction designed to initialize register parameters at the startup boundary. The execution unit extracts the source register address for x0 via the Rs1 port and fetches the constant immediate value of 1 through the secondary operand channel. Because this operation belongs strictly to the I-type command format, the Rs2 field bypasses register-file retrieval and acts directly as a signed

immediate holder. The core computes the arithmetic summation of 0 and 1, scheduling the final data payload of 1 to be committed directly into destination register x18. Driven by the diagnostic multiplexer, all monitored execution variables are output as clean hexadecimal values across the seven-segment displays (HEX7–HEX0) to confirm successful on-chip logic routing.



Figure 6.2: 2nd cycle of combination set

Figure 6.2 illustrates the second execution cycle of the test program, during which another ADDI instruction is performed. The instruction loads the immediate value 3 into register x19. Similar to the previous cycle, this instruction belongs to the I-type format; therefore, the second operand used by the ALU is the immediate value rather than data from a source register. The observed FPGA outputs confirm the correct execution of the instruction and the successful update of register x19.



Figure 6.3: 3rd cycle of combination set

Figure 6.3 shows the third execution cycle of the test program, where an ADD instruction from the R-type instruction format is executed. Unlike the previous ADDI instructions, the ADD instruction requires two register operands. Therefore, the value displayed in Rs2E represents the contents of a source register rather than an immediate value. During this cycle, the processor performs the operation $x20 = x20 + x18$. Since register x18 was initialized with the value 1 in the first cycle and register x20 has not yet been assigned any value, its initial value is assumed to be 0. As a result, the ALU computes the sum as 1, and the output is displayed through the ALU_OutE signal, confirming the correct execution of the instruction.



Figure 6.4: 4th cycle of combination set

Figure 6.4 illustrates the fourth execution cycle of the test program, during which the processor executes a Store Word (SW) instruction belonging to the S-type instruction format. This instruction stores the contents of register x20 into a specified location in data memory. Before the memory write operation can occur, the effective memory address must be calculated. This address is generated by the ALU using the base register and immediate offset provided by the instruction. Consequently, the value displayed in ALU_OutE represents the target memory address used for the store operation. The observed result confirms the correct execution of the address calculation and memory access process.

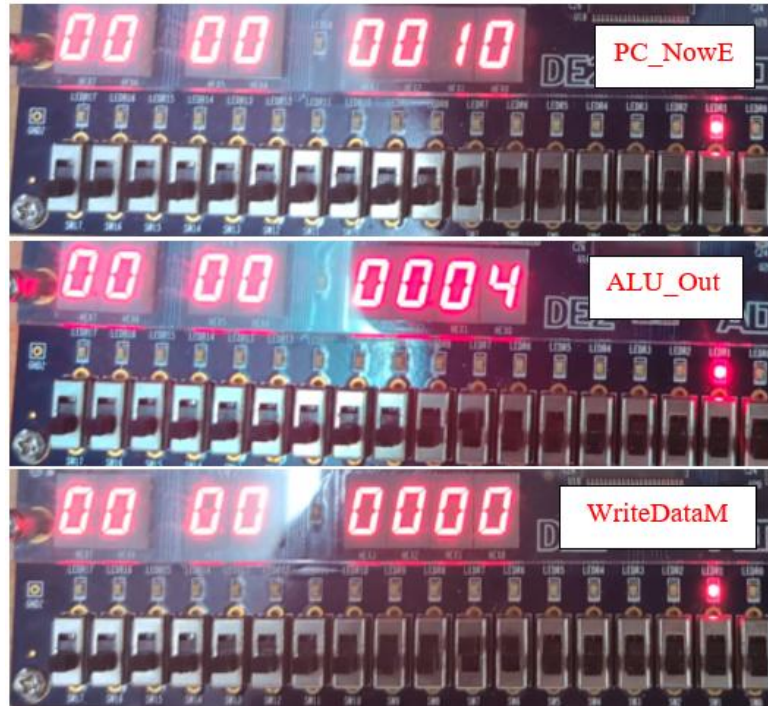


Figure 6.5: 5th cycle of combination set

Figure 6.5 shows the fifth execution cycle, where the processor executes a Load Word (LW) instruction. During this cycle, LEDR1 is asserted, indicating that a load-use hazard has been detected. Since the following instruction requires the loaded data, the hazard detection unit inserts a stall, causing the next cycle to execute a NOP instruction. The value displayed in ALU_OutE represents the memory address used for the load operation. As the Memory stage occurs one cycle after the Execute stage, ReadDataM remains 0 during this cycle and will contain the loaded value in the following cycle.



Figure 6.6: 6th cycle of combination set

Figure 6.6 illustrates the sixth execution cycle, during which the Execute stage processes a NOP instruction inserted by the hazard detection unit. As expected, both PC_NowE and ALU_OutE display a value of 0. Meanwhile, ReadDataM shows the value 1, which corresponds to the data previously stored in memory during the fourth cycle. The loaded value is then written back to register x21, confirming the correct operation of the load and hazard-handling mechanisms.

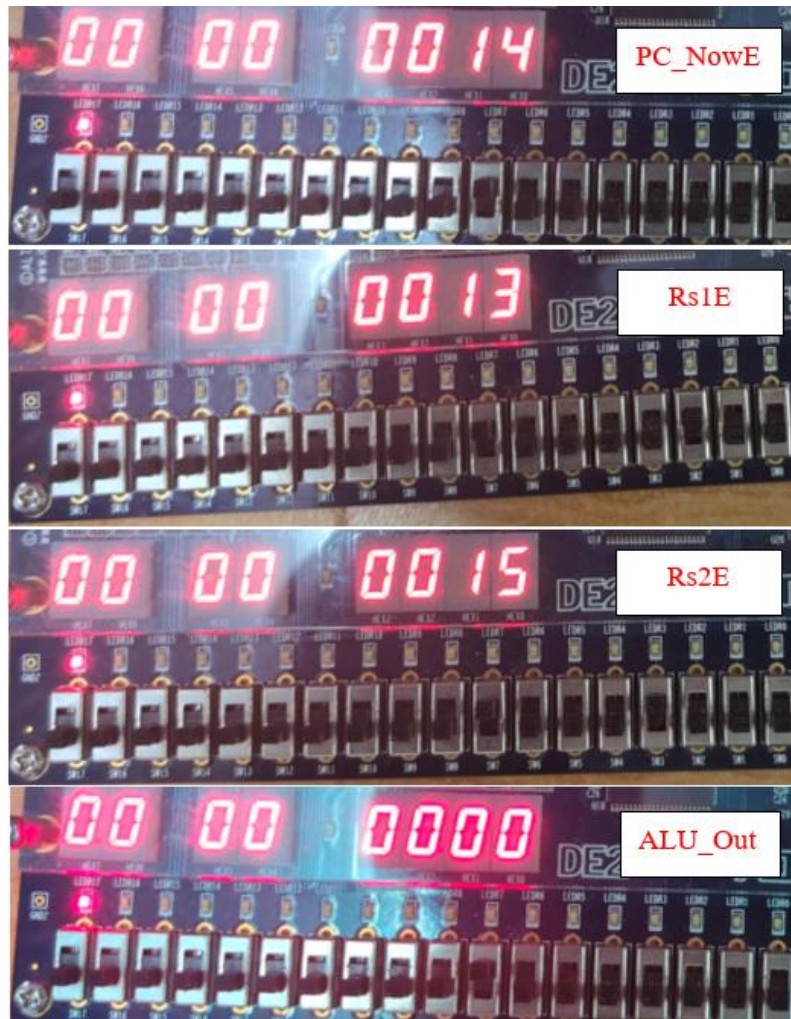


Figure 6.7: 7th cycle of combination set

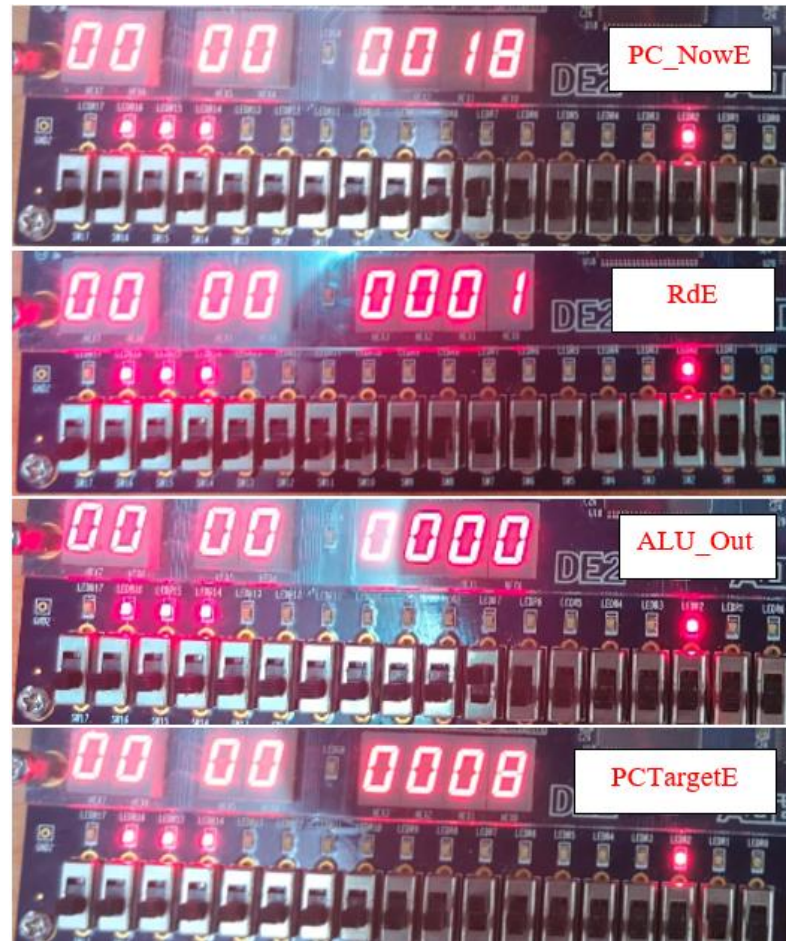


Figure 6.8: 8th of combination set

Figure 6.8 illustrates the eighth execution cycle, where the processor executes a JALR instruction. Since the previous branch instruction was not taken, the value of PC_NowE is 24. The JALR instruction redirects program execution to address 8, where the ADD instruction is located, while simultaneously storing the return address in register x1. During this cycle, LEDR2 is asserted, indicating that the pipeline is flushed to handle the control hazard introduced by the jump instruction. LEDR16 is high, confirming that a jump operation is being executed, while LEDR15 indicates that the instruction is specifically a JALR operation. Additionally, LEDR14 (PCSource) is asserted, showing that the Program Counter selects the jump target address instead of the normal PC+4 path. These observations confirm the correct implementation of jump and control-flow functionality.

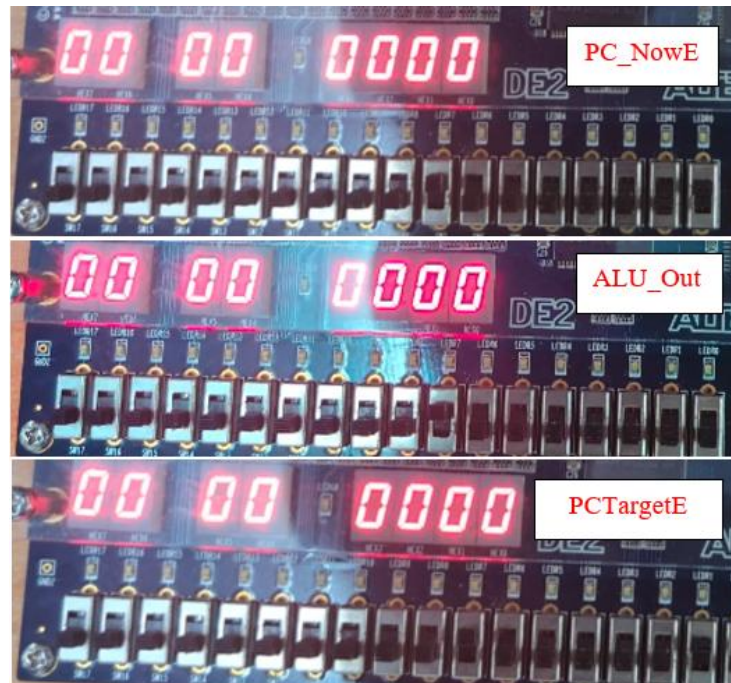


Figure 6.9: 9th and 10th cycle of combination set

Figure 6.9 shows the ninth and tenth execution cycles. During these cycles, the processor executes NOP instructions inserted after the JALR instruction. The control hazard created by the jump operation requires two pipeline flush cycles to prevent incorrect instruction execution. As a result, normal instruction processing resumes only after the hazard has been resolved.

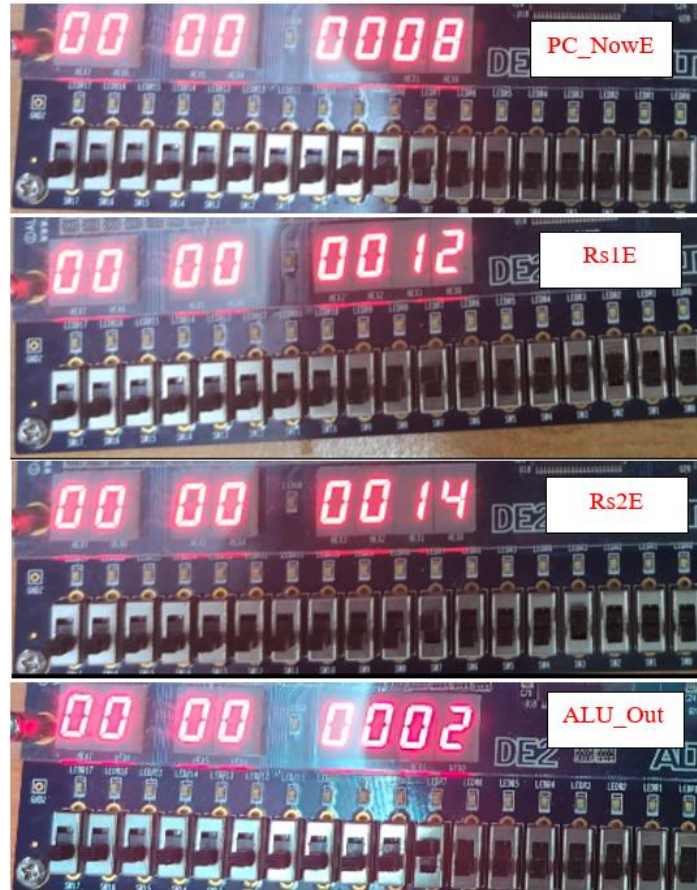


Figure 6.10: 11th cycle of combination set

Figure 6.10 illustrates the eleventh execution cycle, where the processor executes the ADD instruction located at instruction memory address 8. The instruction adds the value in x18 to x20. Since x20 was updated to 1 during the third cycle and x18 contains 1, the ALU produces a result of 2, which is then written back to x20. The processor continues executing this loop until x21 reaches the value 3, at which point the BEQ instruction is taken and program execution jumps to instruction memory address 20.



Figure 6.11: 27th cycle of combination set

Figure 6.11 shows the twenty-seventh execution cycle, during which the processor executes the BEQ instruction. At this point, registers x19 and x21 both contain the value 3, causing the branch condition to evaluate as true. Consequently, the processor performs the branch operation, which is indicated by the assertion of LEDR14 (PCSource) and LEDR2 (Flush). The value of ALU_OutE is 1, confirming that the two registers are equal. The branch target address is 32 (0x20), which is displayed in PCTargetE. Since the branch introduces a control hazard, the following two cycles are flushed and execute NOP instructions.



Figure 6.12: 30th cycle of combination test

Figure 6.12 illustrates the thirtieth and final execution cycle of the test program. During this cycle, the processor executes a JALR instruction that jumps back to its own address, effectively creating an infinite loop to terminate program execution. The return address (0x24) is written to register x1, while both PC_NowE and PCTargetE display the same value, indicating that the jump target is the current instruction address. Similar to the JALR operation observed in Cycle 8, the corresponding control and hazard signals are asserted to handle the resulting control hazard.

6.4 Comparison with Simulation Results

The FPGA observations were compared with the simulation waveforms presented in Chapter V. The hardware outputs matched the expected simulation results for all tested instruction sequences.

The comparison verified:

- Correct Program Counter progression.
- Accurate register-file read operations.
- Correct ALU computation results.
- Successful forwarding operation.
- Proper hazard detection and resolution.
- Reliable pipeline execution.

The consistency between simulation and hardware validation confirms the correctness of the processor architecture and implementation.

6.5 System Appraisal

The developed RV32I processor successfully achieved the objectives established for this project. The processor supports pipelined instruction execution through the five-stage architecture consisting of Instruction Fetch, Instruction Decode, Execute, Memory Access, and Write-Back stages.

The forwarding and hazard detection units effectively manage pipeline hazards while maintaining execution correctness. The FPGA implementation further demonstrates the practical feasibility of the proposed design.

Major achievements of the project include:

- Successful implementation of a 32-bit RV32I pipelined processor.
- Functional forwarding and hazard detection mechanisms.
- Correct execution of supported instructions.
- Successful FPGA deployment and validation.
- Real-time hardware monitoring through seven-segment displays.
- Development of a software translation tool for machine-code generation.

6.6 Limitations and Future Enhancements

Although the processor operates correctly and satisfies the project objectives, several improvements can be considered in future work:

- Support for additional RV32I instructions.
- Implementation of RV32M multiplication and division instructions.
- Integration of instruction and data cache memories.
- Addition of branch prediction mechanisms.
- UART-based program loading and debugging support.
- Enhanced graphical software tools for program generation and analysis.
- Performance optimization for higher operating frequencies.

These enhancements would improve both processor capability and overall system performance.

6.7 Chapter Summary

This chapter presented the FPGA implementation results and hardware validation of the proposed RV32I pipelined processor. Internal datapath signals, including the Program Counter, source register operands, and ALU execution results, were monitored through the FPGA display interface over 30 execution cycles. The observed hardware behavior matched the simulation results, confirming the correctness of the processor architecture, hazard management mechanisms, and overall system implementation.

7. CONCLUSION AND FUTURE WORK

7.1 Conclusion

This project presented the design, implementation, and verification of a 32-bit pipelined RISC-V (RV32I) processor on an FPGA platform. The processor was developed using a five-stage pipeline architecture consisting of Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write-Back (WB) stages. The implemented architecture incorporates essential processor components including the datapath, control unit, arithmetic logic unit (ALU), instruction memory, data memory, forwarding unit, and hazard detection unit. These modules work together to support the execution of RV32I instructions while maintaining correct pipeline operation.

To facilitate processor testing and program generation, a RISC-V Assembly Translation Tool was developed using Python and the Tkinter framework. The software automatically converts assembly instructions into machine code while displaying instruction bit-fields and hexadecimal representations, simplifying the verification process.

Comprehensive verification was conducted through ModelSim simulation and FPGA hardware implementation. Functional testing confirmed correct instruction execution, proper control signal generation, successful hazard handling, and reliable pipeline operation. Experimental results obtained from FPGA testing demonstrated agreement with simulation results, validating the correctness of the proposed design.

Overall, the project successfully achieved its objectives by developing a functional pipelined RV32I processor together with a supporting software toolchain for instruction generation and verification.

7.2 Project Contribution

The major contributions of this project include:

- Design and implementation of a 32-bit RV32I pipelined processor.
- Development of a hazard detection and forwarding mechanism.
- Implementation and verification of the processor on FPGA hardware.
- Development of a Python-based RISC-V Assembly Translation Tool.
- Integration of software and hardware verification workflows.
- Validation through simulation and real-time FPGA testing.

7.3 Future Work

Although the processor operates correctly and satisfies the project objectives, several enhancements can be considered for future development.

7.3.1 Expanded RISC-V Instruction Support

The current implementation focuses primarily on the RV32I base instruction set. Future work may extend support to additional RISC-V extensions such as:

- RV32M (Integer Multiplication and Division)

- RV32C (Compressed Instructions)
- RV32F (Floating-Point Instructions)

7.3.2 Branch Prediction Mechanism

Pipeline performance can be improved through branch prediction techniques. Implementing dynamic or static branch prediction would reduce control hazards and improve instruction throughput.

7.3.3 Cache Memory Integration

The addition of instruction and data caches would significantly reduce memory access latency and improve overall processor performance.

7.3.4 Advanced Debugging Interface

Future versions may incorporate UART communication, on-chip debugging features, or a dedicated monitoring interface to simplify program loading and processor analysis.

7.3.5 Enhanced Software Toolchain

The developed assembly translation tool may be extended to support:

- Complete RV32I instruction coverage
- Assembly file import/export
- Automatic memory image generation
- Integrated simulation support
- Web-based deployment

7.3.6 Performance Optimization

Additional optimization techniques may be applied to improve processor efficiency, including:

- Pipeline balancing
- Improved forwarding strategies
- Reduced critical path delay
- Higher operating frequency

7.4 Final Remarks

The successful implementation of the RV32I pipelined processor demonstrates the practical application of computer architecture concepts, digital system design, and FPGA-based hardware development. The integration of hardware and software components provided valuable insights into modern processor design and established a foundation for future research and development in RISC-V-based embedded systems.

8. REFERENCES

- [1] RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, Version 20240411, 2024.
- [2] David A. Patterson and John L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*, 2nd Edition, Morgan Kaufmann, 2021.
- [3] David A. Patterson and John L. Hennessy, *Computer Architecture: A Quantitative Approach*, 6th Edition, Morgan Kaufmann, 2019.
- [4] [Intel Quartus Prime Lite Edition](#), FPGA Design Software, accessed May 2026.
- [5] ModelSim Intel FPGA Edition, Simulation and Verification Tool, accessed May 2026.
- [6] [Python Software Foundation](#), Python Programming Language Documentation, accessed May 2026.
- [7] [Tkinter Documentation](#), Python GUI Framework Documentation, accessed May 2026.
- [8] [RISC-V International Official Website](#), accessed May 2026.
- [9] Digital Design and Computer Architecture: RISC-V Edition, Sarah L. Harris and David Harris, Morgan Kaufmann, 2021.
- [10] Fundamentals of Digital Logic with Verilog Design, 3rd Edition, McGraw-Hill Education, 2014.